PIMPRI-CHINCHWAD EDUCATION TRUST'S

Pimpri-Chinchwad College of Engineering

(An Autonomous Institute) • Affiliated to Savitribai Phule Pune University (SPPU) • ISO 21001:2018 Certified by TUV

ASSIGNMENT NO 8

Name:	**Siddhesh Parkar**	**Course:**	Design and Analysis of Algorithm
PRN:	**124B1F021**	**Class:**	TY IT A
Topic:	**Travelling Salesman Problem (LC Branch & Bound)**	**Batch:**	A1

1. Problem Statement

SwiftShip Delivery Route Optimizer: A major logistics carrier, SwiftShip, delivers consignments across multiple regional distribution centers. A delivery fleet vehicle must depart from the central warehouse, visit every destination city exactly once, and return to the starting depot while minimizing fuel expenditure and travel duration. Implement the **Least Cost Branch and Bound (LCBnB)** algorithm for the **Travelling Salesman Problem (TSP)** to compute the optimal delivery tour and prune non-promising candidate paths.

2. Course Objectives

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcomes

- Apply branch & bound techniques to solve combinatorial optimization problems.
- Classify P, NP, NP-complete, and NP-Hard problems.

4. Theory

The Travelling Salesman Problem (TSP) is an NP-hard problem seeking a minimum-cost Hamiltonian cycle. While brute-force search evaluates *(N-1)!* permutations, **Least Cost Branch and Bound** systematically structures the search space as a state space tree and prunes branches that cannot possibly beat the best-known solution.

Bounding and Pruning Strategy:

- **Lower Bound Calculation:** For any partial tour, a lower bound on completion cost is estimated by adding the current path cost to the sum of minimum outgoing edge weights from all unvisited nodes.
- **Pruning Rule:** If `lowerBound ≥ bestKnownCost`, the entire subtree is pruned immediately without further exploration.
- **State Space Exploration:** The algorithm explores the most promising states first using a priority queue or depth-first bounding.

Comparison of Approaches:

Approach	Worst-Case Time	Practical Performance
Brute Force TSP	O(N!)	Completely impractical for N > 10
LC Branch and Bound	O(N!) worst case	≪ **O(N!)** due to aggressive lower-bound pruning
Dynamic Programming (Held-Karp)	$O(N^2 2^N)$	Deterministic but high exponential memory

5. Algorithm & Pseudo Code

```
procedure TSP(currentCity, currentCost, currentTime, level):
    if level == n then
        if cost[currentCity][0] != ∞ and currentCost + cost[currentCity][0] < bestCost then
            bestCost ← currentCost + cost[currentCity][0]
            bestTime ← currentTime + timeTaken[currentCity][0]
            Update bestPath with complete Hamiltonian cycle
        end if
        return
    end if
    for nextCity ← 0 to n - 1 do
        if not visited[nextCity] and cost[currentCity][nextCity] != ∞ then
            lowerBound ← currentCost + cost[currentCity][nextCity] + ∑ findMinCost(unvisited)
            if lowerBound < bestCost then
                visited[nextCity] ← true; path[level] ← nextCity
                TSP(nextCity, newCost, newTime, level + 1)
                visited[nextCity] ← false   // Backtrack
            end if
        end if
    end for
end procedure
```

6. Program Source Code (C++)

```
/*
 * Student Name: Siddhesh Parkar
 * PRN: 124B1F021
 * GitHub Repository: https://github.com/siddhesh160606/DAA_lab
 * Assignment 8: Travelling Salesman Problem (TSP) using Least Cost Branch and Bound (SwiftShip)
 */

#include <iostream>
#include <iomanip>

using namespace std;

#define MAX 20
#define INF 999999

int n;
// Distance/cost and time between cities
int cost[MAX][MAX];
int timeTaken[MAX][MAX];

// Best answer
int bestCost = INF;
int bestTime = INF;
int bestPath[MAX + 1];

// Current path
int path[MAX + 1];
bool visited[MAX];

// Calculate minimum outgoing edge from a city
int findMinCost(int city) {
    int minCost = INF;
    for (int i = 0; i < n; i++) {
        if (i != city && cost[city][i] < minCost) {
            minCost = cost[city][i];
        }
    }
    return minCost;
}

// Calculate initial lower bound
int calculateInitialBound() {
    int bound = 0;
    for (int i = 0; i < n; i++) {
        bound += findMinCost(i);
    }
    return bound;
}

// LC Branch and Bound
void TSP(int currentCity, int currentCost, int currentTime, int level) {
    // If all cities are visited
    if (level == n) {
        // Return to starting city
        if (cost[currentCity][0] != INF) {
            int totalCost = currentCost + cost[currentCity][0];
            int totalTime = currentTime + timeTaken[currentCity][0];
            if (totalCost < bestCost) {
                bestCost = totalCost;
                bestTime = totalTime;
                for (int i = 0; i < n; i++) {
                    bestPath[i] = path[i];
                }
                bestPath[n] = 0;
            }
```

```
        }
        return;
    }

    // Try every unvisited city
    for (int nextCity = 0; nextCity < n; nextCity++) {
        if (!visited[nextCity] && cost[currentCity][nextCity] != INF) {
            int newCost = currentCost + cost[currentCity][nextCity];
            int newTime = currentTime + timeTaken[currentCity][nextCity];

            // Simple lower bound
            int lowerBound = newCost;
            for (int i = 0; i < n; i++) {
                if (!visited[i]) {
                    lowerBound += findMinCost(i);
                }
            }

            // Continue only if it can improve current best
            if (lowerBound < bestCost) {
                visited[nextCity] = true;
                path[level] = nextCity;
                TSP(nextCity, newCost, newTime, level + 1);
                visited[nextCity] = false;
            }
        }
    }
}

// Start Branch and Bound
void solveTSP() {
    bestCost = INF;
    bestTime = INF;
    for (int i = 0; i < n; i++) {
        visited[i] = false;
    }

    path[0] = 0;
    visited[0] = true;

    TSP(0, 0, 0, 1);

    if (bestCost == INF) {
        cout << "\nNo possible route exists.\n";
        return;
    }

    cout << "\n========================================\n";
    cout << " OPTIMAL DELIVERY ROUTE\n";
    cout << "========================================\n";
    cout << "Route: ";
    for (int i = 0; i <= n; i++) {
        cout << "City " << bestPath[i] + 1;
        if (i < n)
            cout << " -> ";
    }
    cout << "\n";
    cout << "Minimum Fuel/Travel Cost : " << bestCost << endl;
    cout << "Total Delivery Time : " << bestTime << endl;
    cout << "========================================\n";
}

// Display cost matrix
void displayCostMatrix() {
    cout << "\nCost Matrix:\n\n";
    cout << setw(8) << " ";
    for (int i = 0; i < n; i++)
        cout << setw(8) << "C" << i + 1;
```

```
    cout << endl;

    for (int i = 0; i < n; i++) {
        cout << setw(8) << "C" << i + 1;
        for (int j = 0; j < n; j++) {
            if (cost[i][j] == INF)
                cout << setw(8) << "-";
            else
                cout << setw(8) << cost[i][j];
        }
        cout << endl;
    }
}

// Display time matrix
void displayTimeMatrix() {
    cout << "\nTime Matrix:\n\n";
    cout << setw(8) << " ";
    for (int i = 0; i < n; i++)
        cout << setw(8) << "C" << i + 1;
    cout << endl;

    for (int i = 0; i < n; i++) {
        cout << setw(8) << "C" << i + 1;
        for (int j = 0; j < n; j++) {
            if (timeTaken[i][j] == INF)
                cout << setw(8) << "-";
            else
                cout << setw(8) << timeTaken[i][j];
        }
        cout << endl;
    }
}

// Input data
void inputData() {
    cout << "\nEnter number of cities: ";
    cin >> n;
    if (n < 2 || n > MAX) {
        cout << "Invalid number of cities!\n";
        n = 0;
        return;
    }

    cout << "\nEnter cost matrix:\n";
    cout << "(Enter 0 for same city)\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> cost[i][j];
            if (i == j)
                cost[i][j] = INF;
        }
    }

    cout << "\nEnter time matrix:\n";
    cout << "(Enter 0 for same city)\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> timeTaken[i][j];
            if (i == j)
                timeTaken[i][j] = INF;
        }
    }
    cout << "\nData entered successfully!\n";
}

// Update cost
void updateCost() {
```

```
    int from, to, newCost;
    cout << "\nEnter source city: ";
    cin >> from;
    cout << "Enter destination city: ";
    cin >> to;
    cout << "Enter new cost: ";
    cin >> newCost;

    if (from >= 1 && from <= n && to >= 1 && to <= n) {
        cost[from - 1][to - 1] = newCost;
        cout << "Cost updated successfully!\n";
    }
    else {
        cout << "Invalid city number!\n";
    }
}

// Update time
void updateTime() {
    int from, to, newTime;
    cout << "\nEnter source city: ";
    cin >> from;
    cout << "Enter destination city: ";
    cin >> to;
    cout << "Enter new time: ";
    cin >> newTime;

    if (from >= 1 && from <= n && to >= 1 && to <= n) {
        timeTaken[from - 1][to - 1] = newTime;
        cout << "Time updated successfully!\n";
    }
    else {
        cout << "Invalid city number!\n";
    }
}

int main() {
    cout << "Student Name : Siddhesh Parkar\n";
    cout << "PRN          : 124B1F021\n";
    cout << "GitHub Link  : https://github.com/siddhesh160606/DAA_lab\n";
    cout << "------------------------------------------------------------\n";

    int choice;
    cout << "==========================================\n";
    cout << " SWIFTSHIP DELIVERY ROUTE OPTIMIZER\n";
    cout << " LC BRANCH AND BOUND ALGORITHM\n";
    cout << "==========================================\n";

    do {
        cout << "\n\n--------------- MENU ----------------\n";
        cout << "1. Enter City Data\n";
        cout << "2. Display Cost Matrix\n";
        cout << "3. Display Time Matrix\n";
        cout << "4. Find Optimal Route\n";
        cout << "5. Update Travel Cost\n";
        cout << "6. Update Travel Time\n";
        cout << "7. Exit\n";
        cout << "-------------------------------------\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                inputData();
                break;
            case 2:
                if (n > 0)
                    displayCostMatrix();
```

```
                else
                    cout << "\nPlease enter city data first.\n";
                break;
            case 3:
                if (n > 0)
                    displayTimeMatrix();
                else
                    cout << "\nPlease enter city data first.\n";
                break;
            case 4:
                if (n > 0)
                    solveTSP();
                else
                    cout << "\nPlease enter city data first.\n";
                break;
            case 5:
                if (n > 0)
                    updateCost();
                else
                    cout << "\nPlease enter city data first.\n";
                break;
            case 6:
                if (n > 0)
                    updateTime();
                else
                    cout << "\nPlease enter city data first.\n";
                break;
            case 7:
                cout << "\nThank you for using SwiftShip Route Optimizer!\n";
                break;
            default:
                cout << "\nInvalid choice! Please try again.\n";
        }
    } while (choice != 7);

    return 0;
}
```

7. Sample Execution Output

```
=========================================================
Student Name: Siddhesh Parkar
PRN         : 124B1F021
GitHub Repo : https://github.com/siddhesh160606/DAA_lab
Assignment  : 8 - Travelling Salesman Problem (TSP) using Least Cost Branch and Bound
=========================================================

==========================================
 SWIFTSHIP DELIVERY ROUTE OPTIMIZER
 LC BRANCH AND BOUND ALGORITHM
==========================================


--------------- MENU ----------------
1. Enter City Data
2. Display Cost Matrix
3. Display Time Matrix
4. Find Optimal Route
5. Update Travel Cost
6. Update Travel Time
7. Exit
-------------------------------------
Enter your choice: 1

Enter number of cities: 3

Enter cost matrix:
(Enter 0 for same city)
0 1 2
5 0 9
12 23 0

Enter time matrix:
(Enter 0 for same city)
0 15 20
25 0 20
35 20 0

Data entered successfully!


--------------- MENU ----------------
1. Enter City Data
2. Display Cost Matrix
3. Display Time Matrix
4. Find Optimal Route
5. Update Travel Cost
6. Update Travel Time
7. Exit
-------------------------------------
Enter your choice: 4

======================================
 OPTIMAL DELIVERY ROUTE
======================================
Route: City 1 -> City 2 -> City 3 -> City 1
Minimum Fuel/Travel Cost : 22
Total Delivery Time : 70
======================================
```

```
--------------- MENU ----------------
1. Enter City Data
2. Display Cost Matrix
3. Display Time Matrix
4. Find Optimal Route
5. Update Travel Cost
6. Update Travel Time
7. Exit
-------------------------------------
Enter your choice: 7

Thank you for using SwiftShip Route Optimizer!
```

8. Conclusion

The Least Cost Branch and Bound algorithm provides an exact, guaranteed optimal solution for the Travelling Salesman Problem while avoiding brute-force combinatorial explosion. Through dynamic lower bound estimation, the search tree is aggressively pruned, making exact fleet routing for regional delivery hubs practical, cost-effective, and fuel-efficient.

GitHub Repository Link

Repository Link: https://github.com/siddhesh160606/DAA_lab